

MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring Requirements Engineering

Michał Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
`office@pads.rwth-aachen.de`

Abstract. This document outlines the requirements engineering phase for a time-series-enhanced predictive process monitoring pipeline. The primary objective of this project is to assess the impact of incorporating time series data on the predictive performance of the pipeline compared to a standard event-log-based model. The system is designed to predict the final outcome of a loan application and then estimate the remaining processing time for ongoing cases. To achieve this, we have identified and documented detailed functional and non-functional requirements, including data preprocessing, feature engineering, model training, evaluation, and system infrastructure. The document also identifies key stakeholders and technical constraints and creates a functional model with clear team responsibilities to meet the lab's evaluation criteria.

Keywords: Predictive Process Monitoring · Requirements Engineering
· Time-Series Analysis · Process Mining · Feature Engineering

1 Stakeholders and Constraints

1.1 Main Stakeholders

To ensure the successful delivery of the time-series-enhanced predictive process monitoring pipeline, we have identified the following primary stakeholders:

- **Course Instructors and Teaching Assistants:** Prof. Willibrordus Martinus Pancratius van der Aalst, Alessandro Berti, Nina Graves, and Viki Peeva. They define the success criteria and evaluate the project.
- **Group 5:** Konstantinos, Roman and Michał. Responsible for the design, implementation, and testing of the pipeline.
- **Process Owners:** Simulated stakeholders who represent the business users benefiting from the predictive pipeline.

1.2 Project Constraints

The project's design and implementation are constrained by several strict technical and organizational factors.

- **Python, PM4Py, scikit-learn:** The pipeline must be implemented in Python, utilizing PM4Py for process integrations and scikit-learn for machine learning.
- **Version Control:** The codebase must be publicly hosted and maintained on GitHub.
- **Limited Computation:** The dataset [1] contains 1.2 million events requiring memory-efficient implementations.
- **Limited Time:** Development is strictly bound by the semester deadlines.

2 Requirements Specification

2.1 Data & Preprocessing

- **ID:** REQ-DP-01
- **Type:** Functional
- **Text:** The system shall be able to import and parse event logs in the XES (eXtensible Event Stream) format into an internal data structure suitable for downstream process mining and machine learning tasks.

- **ID:** REQ-DP-02
- **Type:** Functional
- **Text:** The system shall preprocess event log and time-series data by:
 - Detecting and handling missing values using configurable strategies (e.g., imputation, forward-fill, removal).
 - Resampling time-series data to a uniform temporal resolution based on a configurable interval.
 - Preserving event ordering and timestamp consistency during preprocessing.

- **ID:** REQ-DP-03
- **Type:** Functional
- **Text:** The system shall construct case prefixes from event logs by:
 - Generating prefix sequences for each case up to a configurable maximum length.
 - Supporting prefix generation strategies (e.g. growing prefix, fixed-length sliding window).
 - Preserving temporal order and associated event attributes.

- **ID:** REQ-DP-04
- **Type:** Functional
- **Text:** The system shall extract features from event logs, specifically:
 - Extract control-flow features (e.g., activity sequences, n-grams, directly-follows relations)
 - Extract temporal features (e.g., inter-event times, case durations, waiting times)
 - Allow users to configure:

- * The selection of feature types used for downstream ML tasks
 - * Parameters of the selected features (e.g., n for n-grams, aggregation methods)
- **ID:** REQ-DP-05
 - **Type:** Functional
 - **Text:** The system shall extract features from time-series data, specifically:
 - Apply sliding-window techniques with configurable window size and step to segment the data.
 - Compute statistical features for each window, including mean, variance, minimum, maximum, and trend (e.g., linear slope).
 - Align extracted time-series features with event-based data using timestamps.
 - **ID:** REQ-DP-06
 - **Type:** Functional
 - **Text:** The system shall align event log-derived features and time-series features, specifically:
 - Synchronize features using timestamps as the primary alignment key.
 - Ensure that each aligned feature vector corresponds to a single case prefix or case instance.
 - Handle temporal mismatches and missing alignments using configurable strategies, including interpolation, truncation, or padding.

2.2 Modelling & Evaluation

- **ID:** REQ-ME-01
- **Type:** Functional
- **Text:** The system shall implement at least one predictive process monitoring task based on event log data, from the following list:
 - Outcome prediction (classification of process instance results).
 - Remaining time prediction (regression of time until case completion).
- **ID:** REQ-ME-02
- **Type:** Functional
- **Text:** For each prediction task, the system shall train baseline models using only event log data. Specifically:
 - Use only event log data and derived case prefixes as input features.
 - Explicitly exclude all time-series-derived features.
 - Produce predictions as defined in REQ-ME-01.
- **ID:** REQ-ME-03
- **Type:** Functional
- **Text:** For each prediction task defined in REQ-ME-01, the system shall extend the baseline models defined in REQ-ME-02 by incorporating time-series data. Specifically:

- The extended models shall use all features defined in REQ-ME-02 (event-log data and derived case prefixes).
- The extended models shall additionally include time-series-derived features (e.g., sliding-window statistics and sensor measurements).
- Time-series features shall be aligned with event log prefixes using consistent temporal references.
- The extended models shall produce predictions according to the same task definition as their corresponding baseline model.

– **ID:** REQ-ME-04

– **Type:** Functional

– **Text:** The system shall evaluate predictive models using suitable metrics, including at least the following:

- For classification tasks the provided metrics should include at least accuracy and AUC.
- For regression tasks the provided metrics should include at least MAE and RMSE.

Evaluation metrics shall be computed on a held-out test set consistent across all compared models.

– **ID:** REQ-ME-05

– **Type:** Functional

– **Text:** The system shall use time-based data splitting strategies (e.g., chronological split or rolling-origin evaluation) for training, validation, and testing in order to preserve temporal ordering and prevent information leakage.

– **ID:** REQ-ME-06

– **Type:** Functional

– **Text:** The predictive models shall be implemented using scikit-learn-compatible pipeline structures. Specifically:

- Each model shall be encapsulated in a pipeline that supports sequential processing steps (e.g., preprocessing, feature extraction, and prediction).
- Each pipeline shall implement a standard scikit-learn interface, including `fit` and `predict` methods.
- Hyperparameters shall be optimized using a systematic validation strategy such as grid search or random search.

2.3 Infrastructure & Tooling

– **ID:** REQ-IT-01

– **Type:** Non-Functional

– **Text:** Experiments shall be strictly reproducible by utilizing fixed random seeds and centralized configuration files. Experiments with identical configuration and dataset shall produce identical results.

– **ID:** REQ-IT-02

- **Type:** Non-Functional
- **Text:** The code base shall be modular, with separate components for data processing, feature engineering, model training, and evaluation.
- **ID:** REQ-IT-03
- **Type:** Non-Functional
- **Text:** Automated tests shall be provided and executable via a single command (e.g., pytest). The tests shall cover core components including log loading, case prefix generation, and feature extraction using a synthetic dataset.
- **ID:** REQ-IT-04
- **Type:** Non-Functional
- **Text:** All system dependencies shall be explicitly documented using a dependency management system to ensure reproducibility and enable execution of the full pipeline.
- **ID:** REQ-IT-05
- **Type:** Non-Functional
- **Text:** The system shall be extensible such that new feature types, models, or evaluation metrics can be integrated without modifying existing components.
- **ID:** REQ-IT-06
- **Type:** Non-Functional
- **Text:** All non-trivial functions and classes must contain descriptive doc-strings.

2.4 Reporting & Explainability

- **ID:** REQ-RE-01
- **Type:** Functional
- **Text:** The system shall produce a comparative evaluation of baseline and extended models for each prediction task, including:
 - Performance differences across all evaluation metrics defined in REQ-ME-04.
 - Performance as a function of prefix length.
- **ID:** REQ-RE-02
- **Type:** Functional
- **Text:** The system shall compute and visualize feature importance scores for each trained model, comparing the relative contribution of log-based features versus time-series features to predictive performance.

2.5 Documentation

- **ID:** REQ-DOC-01
- **Type:** Non-Functional

- **Text:** The repository shall include a README containing installation instructions, dataset placement instructions, and a guide to running the full pipeline end-to-end.
- **ID:** REQ-DOC-02
- **Type:** Non-Functional
- **Text:** The project shall include a short technical document (3–5 pages) describing the system architecture, data flow from raw inputs to predictions, and the main design choices (encodings, models, hyperparameters). A simple architecture diagram should be included.
- **ID:** REQ-DOC-03
- **Type:** Non-Functional
- **Text:** The final report (10–15 pages, excluding references and appendices) shall cover:
 - Data preprocessing and feature engineering choices
 - Model specification and evaluation protocol
 - A critical discussion of the added value of time-series features, including potential reasons for success or failure, computational overhead, and generalizability of findings
- **ID:** REQ-DOC-04
- **Type:** Non-Functional
- **Text:** The report shall include a comparative analysis of baseline and enhanced models, supported by at least qualitative significance assessment (e.g. confidence intervals, boxplots across runs, or a careful discussion of observed differences).

3 Allocation and Flow-Down

3.1 High-Level Goals

The primary objective is to evaluate whether incorporating external time-series data improves predictive process monitoring over a standard event-log baseline. To achieve this, the pipeline will predict loan application outcomes and estimate remaining processing times for ongoing cases

3.2 The Functional Model

The system is structured as a five-stage pipeline. Each stage is defined by its inputs and outputs, and the implementation details are captured in the requirements specification. However, the exact internal data representations between stages are not yet defined and will be determined during implementation.

F1: Data ingestion

Input: Raw event log (XES), raw time-series sources.

Output: Cleaned event log, preprocessed time-series data. Both split into test/train set.

F2: Feature Engineering

Input: Cleaned event log, preprocessed time-series data.

Output: Labelled feature vectors per case prefix (log-based and time-series features).

F3: Model Training

Input: Labelled feature vectors (training and validation sets).

Output: Trained baseline and extended models with tuned hyperparameters.

F4: Evaluation

Input: Trained models, held-out test set.

Output: Evaluation metrics per model and prefix length.

F5: Reporting

Input: Evaluation metrics, trained models.

Output: Comparative visualizations, feature importance plots, final report.

3.3 Module Allocation

Table 1 maps each system module to the requirements it satisfies and the responsible team member.

Module	Owner	Requirements
preprocessing/	Roman	REQ-DP-01, REQ-DP-02
features/	Konstantinos	REQ-DP-03, REQ-DP-04
features/	Michał	REQ-DP-05, REQ-DP-06
models/	Konstantinos	REQ-ME-01, REQ-ME-02
models/	Roman	REQ-ME-03, REQ-ME-06
evaluation/	Michał	REQ-ME-04, REQ-ME-05
evaluation/	Konstantinos	REQ-RE-01, REQ-RE-02
tests/	Roman	REQ-IT-03
config/	Roman	REQ-IT-01
docs/	Michał	REQ-DOC-01, REQ-DOC-02

Table 1. Module allocation and ownership.

Non-functional requirements REQ-IT-02, REQ-IT-05 and REQ-IT-06 apply across all modules and are the shared responsibility of the entire team. The final report (REQ-DOC-03, REQ-DOC-04) is likewise a shared deliverable.

3.4 Requirements Management

The requirements specified in this document will remain stable. Changes to high-level requirements will only occur in exceptional circumstances (e.g., a fundamental shift in the prediction task) and will be explicitly documented. Detailed sub-requirements identified during development will be tracked as GitHub issues linked to their parent requirement, rather than requiring updates to this document.

References

1. van Dongen, B.F.: BPI Challenge 2017. Eindhoven University of Technology (2017). <https://research.tue.nl/en/datasets/bpi-challenge-2017>